TransitOps PRD

Product Requirements Document (PRD)

TransitOps — Fleet & Logistics Operations Platform

Version: 1.0 **Prepared for:** Odoo Hackathon 2026 **Document type:** Product Requirements Document **Status:** Based on current implementation (backend + frontend codebase review)

---

1. Overview

1.1 Purpose

TransitOps is a fleet and logistics operations platform that lets a transport/logistics company manage vehicles, drivers, trips, fuel, maintenance, and cost reporting from a single system. It replaces manual/spreadsheet-based fleet tracking with a role-based web application backed by a REST API.

1.2 Problem Statement

Small-to-mid-size fleet operators lack a lightweight, purpose-built system to: - Assign the right vehicle and driver to a trip without double-booking or overloading. - Track vehicle health and enforce maintenance cycles. - Track fuel and operational costs per vehicle/trip. - Give different roles (dispatch, safety, finance, management) visibility appropriate to their job.

1.3 Target Users / Roles

The system enforces Role-Based Access Control (RBAC) with five roles:

Role	Primary Responsibility
`fleet_manager`	Vehicle master data, full oversight, reporting
`dispatcher`	Trip creation, dispatch, completion, cancellation
`safety_officer`	Driver master data (licenses, KYC), maintenance records
`financial_analyst`	Fuel logs, expenses, cost/performance reports
`viewer`	Read-only access

1.4 Demo Accounts (seeded)

- Fleet Manager — `manager@transitops.com`
- Dispatcher — `dispatcher@transitops.com`
- Safety Officer — `safety@transitops.com`
- Financial Analyst — `finance@transitops.com`

---

2. Goals & Non-Goals

2.1 Goals

- Prevent operational errors at the data layer: overloaded vehicles, expired-license drivers, double-booked assets, retired/in-shop vehicles being dispatched.
- Provide a full trip lifecycle: draft → assigned/dispatched → in_transit/delivered → completed (or cancelled).
- Give each role a focused, permissioned UI and API surface.
- Provide near-real-time dashboards and exportable reports for cost and performance analysis.
- Maintain an audit trail of create/update/delete/restore actions on core entities.

2.2 Non-Goals (not in current scope)

- Live GPS/telematics integration (the “tracking” and “AI route optimization” pages exist as stubs/placeholders, not fully wired to live data).
- Payroll processing (driver `paymentType` / `paymentAmount` are recorded but not processed).
- Customer-facing booking portal.
- Native mobile app (frontend is a responsive web app only).

---

3. System Architecture (as built)

- **Backend:** Node.js + Express, MongoDB via Mongoose, JWT authentication, Redis-style in-memory response caching (`cacheMiddleware`), Winston logging, field-level encryption for sensitive driver PII.
- **Frontend:** Vanilla HTML/CSS/JavaScript (no framework/build step), glassmorphism UI, module-per-page JS (`trips.js`, `vehicles.js`, `drivers.js`, etc.), a shared `api.js` service and `auth.js` for session handling, offline-queue support (`offline.js`).
- **Infrastructure:** Dockerfile, `docker-compose.yml`, Kubernetes manifests (`k8s/deployment.yaml`, `k8s/service.yaml`), Nginx config, PM2 `ecosystem.config.js`, GitHub Actions CI (`.github/workflows/ci.yml`).
- **Data layer pattern:** Repository pattern for Trip (`tripRepository`) with a base repository/service abstraction reused across entities.

---

4. Core Entities (Data Model Summary)

4.1 User

`name`, `email` (unique), `phone`, `passwordHash` (bcrypt, cost factor 12), `role` (`fleet_manager` | `dispatcher` | `safety_officer` | `financial_analyst` | `viewer`).

4.2 Vehicle

`name`, `registrationNumber` (unique), `vehicleType` (truck/van/bus/pickup/trailer/tanker), `fuelType` (diesel/gasoline/cng/electric/hybrid), `maxLoadCapacity` (kg), `odometer`, `acquisitionCost`, `maintenanceInterval`, `nextMaintenanceMileage`, `region`, `status` (available/on_trip/in_shop/retired).

4.3 Driver

`name`, `licenseNumber` (unique, **field-encrypted**), `licenseCategory` (A–E), `licenseExpiryDate`, `contactNumber` (**field-encrypted**), `safetyScore` (0–100, default 100), `status` (available/on_trip/off_duty/suspended), `paymentType` (monthly/per_trip), `paymentAmount`, `kyc.aadharNumber` + `kyc.aadharVerified`.

4.4 Trip

`tripCode` (auto-generated, e.g. `TRIP-A1B2C3D4`), `source`, `destination`, `vehicle` (ref), `driver` (ref), `cargoWeight`, `plannedDistance`, `actualDistance`, `fuelConsumed`, `revenue`, `scheduledDate`, `eta`, `completedAt`, `priority` (low/medium/high/urgent), `status` (draft/assigned/loading/dispatched/in_transit/delivered/completed/delayed/cancelled).

4.5 FuelLog

`vehicle`, `trip` (optional), `date`, `liters`, `costPerLiter`, `cost`, `station`.

4.6 Expense

`vehicle`, `trip` (optional), `expenseType` (toll/insurance/tax/fine/parking/other), `amount`, `date`, `description`.

4.7 Maintenance

`vehicle`, `description`, `maintenanceType` (preventive/corrective/inspection/emergency), `cost`, `dateStart`, `dateEnd`, `status` (open/closed), `notes`.

4.8 Notification

`userId` (optional, supports broadcasts), `title`, `message`, `type` (info/warning/error/success), `read`, `referenceId`.

4.9 AuditLog

`action` (CREATE/UPDATE/DELETE/RESTORE), `modelName`, `documentId`, `userId`, `changes`.

---

5. Functional Requirements

5.1 Authentication & Authorization

- FR-1: Users register/log in via email + password; passwords stored as bcrypt hashes (cost 12).
- FR-2: JWT-based session with refresh token endpoint (`/api/v1/auth/refresh`) and forgot/reset password flow.
- FR-3: `GET /auth/me` returns the current authenticated user's profile.
- FR-4: Every protected route requires a valid JWT (`protect` middleware); role-restricted routes additionally require `authorize(...)`.

5.2 Vehicle Management (Fleet Manager)

- FR-5: CRUD on vehicles, restricted to `fleet_manager` for create/update/delete; all authenticated roles can read.
- FR-6: Vehicle list/detail responses are cached for 300 seconds and invalidated on write.
- FR-7: Vehicle `status` drives trip eligibility (see Business Rules, §6).

5.3 Driver Management (Safety Officer)

- FR-8: CRUD on drivers, restricted to `safety_officer` for create/update/delete; all authenticated roles can read.
- FR-9: Sensitive fields (`licenseNumber`, `contactNumber`) are encrypted at rest.
- FR-10: Driver `status` and `licenseExpiryDate` gate trip assignment/dispatch eligibility.

5.4 Trip Lifecycle Management (Dispatcher)

- FR-11: Create a trip in `draft` status, specifying vehicle, driver, cargo weight, source/destination, schedule window.
- FR-12: Edit a trip only while it is in `draft` status.
- FR-13: Dispatch a `draft` trip → sets vehicle and driver to `on_trip`, trip moves to `dispatched`.
- FR-14: Complete an active trip (`dispatched` / `in_transit` / `delivered`) with actual distance, fuel consumed, and revenue → releases vehicle and driver back to `available`, updates vehicle odometer, sets `completedAt`.
- FR-15: Cancel a trip while `draft` or `dispatched` → releases vehicle/driver if they had been reserved.
- FR-16: Generic status update endpoint (`PUT /trips/:id/status`) enforces valid transitions and automatically frees/reserves vehicle and driver based on the transition.
- FR-17: List trips with filters: status, priority, vehicle, driver, date range, free-text search on trip code; paginated.
- FR-18: Every trip create/dispatch/complete/cancel/status-change action writes an `AuditLog` entry.

5.5 Fuel & Expense Tracking (Financial Analyst)

- FR-19: Record fuel logs (liters, cost, station, optional trip link).
- FR-20: Record operating expenses by type (toll, insurance, tax, fine, parking, other), optionally linked to a vehicle/trip.
- FR-21: "Live rates" endpoint for current fuel pricing reference.
- FR-22: Only `financial_analyst` and `fleet_manager` may create fuel logs/expenses; all authenticated roles may view.

5.6 Maintenance Management (Fleet Manager / Safety Officer)

- FR-23: Open a maintenance record against a vehicle (blocked if the vehicle is currently on a trip or retired); vehicle status set to `in_shop`.
- FR-24: Update an open maintenance record.
- FR-25: Close a maintenance record (blocked if already closed); returns vehicle to an operable status.

5.7 Reporting & Analytics

- FR-26: Dashboard KPIs endpoint (`/reports/dashboard`) — fleet/trip/cost summary metrics, cached 5 minutes.
- FR-27: Fuel efficiency report per vehicle.
- FR-28: Operational cost report — combines fuel, expenses, maintenance cost, and revenue per vehicle.
- FR-29: Driver performance report, restricted to `financial_analyst` and `fleet_manager`.

- FR-30: CSV export of trips, restricted to `financial_analyst` and `fleet_manager`.

5.8 Notifications

- FR-31: System auto-generates a “Maintenance Due Soon” notification when a vehicle’s odometer comes within 500 km of `nextMaintenanceMileage` upon trip completion.
- FR-32: Users can list their notifications and mark individual notifications as read.

5.9 Frontend Pages (implemented)

Login, Dashboard, Trips, Vehicles, Drivers, Fuel & Expense, Maintenance, Reports, Analytics, Tracking (map/placeholder), plus shared error page.

---

6. Business Rules (enforced in code)

These are the validation rules currently implemented in `tripService.js` and `maintenanceController.js`, referenced by ID in error messages:

Rule ID	Rule
BR-1	A trip cannot be created against a non-existent or **retired** vehicle.
BR-2	A trip cannot be created against a non-existent driver.
BR-3	A trip can only be dispatched if the vehicle status is `available`.
BR-4	A trip cannot be assigned/dispatched to a **suspended** driver or a driver with an **expired license**.
BR-5	Cargo weight must not exceed the assigned vehicle’s `maxLoadCapacity`.
BR-9	A maintenance record cannot be opened on a vehicle that is currently **on a trip** or **retired**; opening one sets the vehicle to `in_shop`.
Scheduling conflict	A vehicle or driver cannot be double-booked: overlapping `scheduledDate` – `eta` windows on active (non-completed/cancelled) trips are rejected, with the conflicting trip code returned in the error.
Terminal state	Once a trip is `completed` or `cancelled`, its status can never change again.
Draft-only edit	A trip’s core details (vehicle, driver, schedule, etc.) can only be edited while it is in `draft` status.
Maintenance close	A maintenance record that is already `closed` cannot be closed again.

---

7. Non-Functional Requirements

- NFR-1 (Security): JWT auth on all protected endpoints; RBAC per endpoint; bcrypt password hashing; field-level encryption for driver PII (license number, contact number).
- NFR-2 (Performance): GET-heavy endpoints (vehicles, drivers, reports) are cached for 300s via `cacheMiddleware` and invalidated on writes via `clearCache`.
- NFR-3 (Auditability): All state-changing trip actions are logged to `AuditLog` with actor, action, and change payload.
- NFR-4 (Observability): Centralized Winston logging (`combined.log`, `error.log`); structured log messages for trip lifecycle events.
- NFR-5 (Resilience/Offline): Frontend includes an offline request queue (`offline.js`) for degraded connectivity.
- NFR-6 (Deployability): Containerized via Docker; deployable to Kubernetes (Deployment + Service manifests); Nginx reverse proxy config included; CI pipeline via GitHub Actions.
- NFR-7 (Data integrity): Unique constraints on vehicle registration number, driver license number, and user email; Mongoose-level required-field and enum validation on all entities.

8. API Surface Summary

Base path: `/api/v1`

Resource	Endpoints
Auth	`POST /auth/register`, `POST /auth/login`, `POST /auth/refresh`, `POST /auth/forgot-password`, `POST /auth/reset-password`, `GET /auth/me`
Vehicles	`GET/POST /vehicles`, `GET/PUT/DELETE /vehicles/:id`
Drivers	`GET/POST /drivers`, `GET/PUT/DELETE /drivers/:id`
Trips	`GET/POST /trips`, `PUT /trips/:id/dispatch`, `PUT /trips/:id/complete`, `PUT /trips/:id/cancel`, `PUT /trips/:id/status`
Fuel & Expense	`GET /fuel-expense/live-rates`, `GET/POST /fuel-expense/fuel`, `GET/POST /fuel-expense/expenses`
Maintenance	`GET/POST /maintenance`, `GET/PUT /maintenance/:id`, `PUT /maintenance/:id/close`
Reports	`GET /reports/dashboard`, `GET /reports/fuel-efficiency`, `GET /reports/operational-cost`, `GET /reports/driver-performance`, `GET /reports/export/trips`
Notifications	`GET /notifications`, `PUT /notifications/:id/read`
AI (stub)	`POST /ai/optimize-route`, `POST /ai/assistant`

(Full request/response contracts, status codes, and validation schemas belong in the companion TRD.)

9. Out-of-Scope / Known Gaps (for roadmap)

- `ai/optimize-route` and `ai/assistant` routes exist but are placeholder handlers — not yet backed by a real optimization/LLM service.
- The Tracking page has no live GPS feed integration yet.
- No soft-delete/restore UI is exposed for `AuditLog`'s `RESTORE` action, though the audit model supports it.
- `constants/index.js` defines a broader `ROLES` list (`admin`, `driver`, `analyst`) than what is currently enforced in route-level `authorize()` calls — should be reconciled in a future cleanup.

10. Success Metrics (suggested)

- Zero double-booked vehicle/driver incidents post-launch (enforced by scheduling-conflict rule).
- % reduction in overdue maintenance events (via the 500 km early-warning notification).
- Trip completion data captured (distance/fuel/revenue) for ≥95% of completed trips, enabling accurate cost reporting.
- Dashboard/report load time under cache-warm conditions < 300ms (cache TTL 300s).

Next document in this series: Technical Requirements Document (TRD) — say "trd" to generate.